

Parametric Design Learning

S. Lin

*M.S. Aeronautics and Astronautics
Stanford University*

Abstract - I present Parametric Design Learning (PDL) as an exploration of reinforcement learning techniques in the domain of engineering component design. I begin by discussing the current 3D object design environment, and I provide a motivation for the utility of models that can learn the sequential nature of parametric object design. Next, I state my problem and define the relevant spaces and hyper-parameters. I rationalize the training data I used as well as describe the algorithms and methods employed. Finally, I present and analyze the results, identify key training hyper-parameters, and explain their effects on the output and convergence.

I. Introduction

3D object modeling is widely used in the field of engineering to create parts with a rigorously defined geometry. An example of such a part is a simple screw with a particular pitch, diameter, and head. The format of the 3D engineering part generated is unique from other forms of 3D-object data. For example, an object in animation, which exists for aesthetic purposes, is represented by a set of points in 3D space with surfaces interpolated between them (meshes). Meshes are imprecise and impractical for engineering purposes where part dimensions need to be exact, modifiable, and constrainable.

As such, engineering parts are designed parametrically through a set of sequential operations such as sketching, extruding, and cutting. Each of these operations takes a unique set of parameters. For example, the creation of a box requires an x, y, and z dimension as parameters. Figure 1 demonstrates the design horizon of a screw.

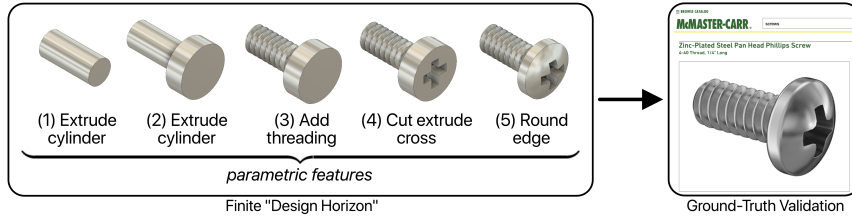


Fig. 1 Sequence of parameterized operations (design policy) typically executed to design a screw

Currently, engineering part design is done manually. However, inspired by the capabilities of generative AI in the realm of image creation, I decided to explore the possibility of extending these capabilities from a 2D to 3D space. The caveat lies in the requirement that the generated object also be parametric. 2D image data is strikingly similar to non-parametric 3D data in that both are an array of points that take a set of values (RGB for images and a binary occupancy indicator for 3D objects). As such, this project assumes that generative AI capabilities can be reasonably extended to non-parametric 3D objects (elaborated on in the Related Work section). PDL seeks to use reinforcement learning techniques to learn the design procedure of a parametric object from its non-parametric equivalent.

II. Related Work

Much of the current work that lies at the intersection of learning and 3D object modeling concerns the use of Neural Radiance Fields (NeRFs) [1]. These are deep networks designed to simulate camera rays in a continuous space. Recent works such as DreamFusion [2] and Magic3D[3] by NVIDIA have studied the generation of novel NeRFs using a text-to-image diffusion process. However, novel generation of parametric designs remain an under-explored topic. The ABC dataset [4] of geometric CAD parts has prompted several initial works on parametric feature recognition. However, there is still a disconnect between feature-based classification of parametric 3D models and the learning of their design

policy.

On the reinforcement learning front, techniques have been developed to tackle both continuously-parameterized action spaces [5], as well as dynamically-sized action spaces, both of which are partially applicable to the problem of learning a parametric design policy, though no particular progress has been made in the specific direction until now.

III. PDL

A. Problem Statement

The problem that PDL solves is learning an optimal sequence of parameterized operations that seeks to efficiently fit a parametric 3D object output with a non-parametric target part voxel. A voxel is an occupancy grid, taking form of a 3D binary array. Each element inside a voxel corresponds with a coordinate in continuous space. If that coordinate is occupied by the object in continuous space, the voxel element = 1. If not, the voxel element = 0.

This problem is a sequential problem that can be structured as a Markov Decision Process (MDP) defined below.

- $|S|$: The state space of this problem is inherently continuous. In order to simplify this state space, I limited the space to the size of the minimum bounding box of the target object and discretized this bounding box uniformly with a fixed resolution into a voxel. The initial state s_i is an empty voxel of 0s. The target state s_t is the voxel of the target object.
- $|A|$: The action space of this problem is inspired by the capabilities of common CAD software. I down-sampled an otherwise complex action space to only contain three primitive actions:
 - 1) Grow a box from a given centroid with parameterized x, y, z dimensions.
 - 2) Grow a cylinder aligned with a cardinal axis from a given centroid with parameterized radii and height.
 - 3) Grow a sphere from a given centroid with parameterized radii.
- π : A policy is defined as a sequence of actions a from a starting empty voxel state s_i to a final voxel state s_f .
- $R(s|\beta)$: The reward function of this problem is defined to be the following:

$$\begin{aligned}
 s_n[x, y, z] == s_t[x, y, z] == 1 &\Rightarrow R(s_n[x, y, z]) = 1 \\
 s_n[x, y, z] == s_t[x, y, z] == 0 &\Rightarrow R(s_n[x, y, z]) = 0 \\
 s_n[x, y, z] \neq s_t[x, y, z] &\Rightarrow R(s_n[x, y, z]) = \beta : \beta \leq -1 \\
 R(s) &= \sum_{x,y,z} R(s[x, y, z])
 \end{aligned}$$

$R(s)$ rewards coordinates in s and s_t that are both occupied, is neutral towards coordinates that are both unoccupied, and penalizes coordinates that disagree. The extent of penalization is hyper-parameterized by β . Figure 2 demonstrates a 2D example of how the reward function is applied.

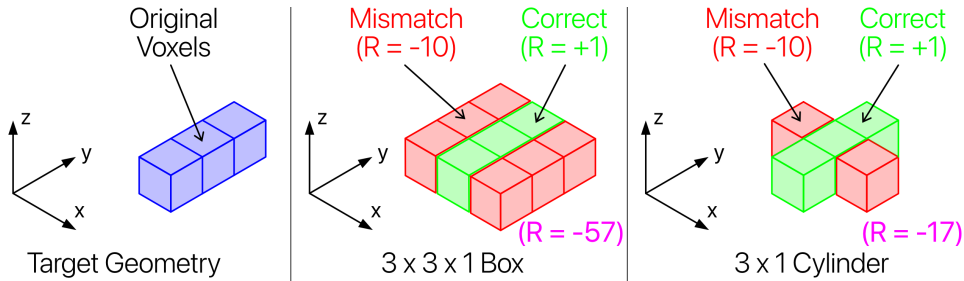


Fig. 2 Demonstration of how rewards are calculated based on a target voxel where $\beta = -10$.

It is important to note that this problem does not require an observation-space $|O|$ or state-transition probabilities $T(s'|s, a)$. There is no state uncertainty, and given a current state and an action, the next state is deterministic.

B. Data Selection

Three target parts were selected based on the limited scope of $|A|$. Since $|A|$ only contains actions for creating three primitive geometries, the 'easy' geometry was selected to be a simple combination a cylinder and box. Even with a

limited $|A|$, the model should design the 'easy' geometry perfectly. The 'medium' geometry was selected to be a thumb screw. This geometry contains complex surface features such as threads and a knurled finish that make a perfect fit with only three primitive actions impossible. Nonetheless, it is primarily composed of 2 concentric cylinders which my model should be able to capture. Lastly, the 'hard' geometry was selected to be a 3D star. The star is not intuitively composed of either boxes, cylinders, or spheres, hence how the model learns to build it is not immediately obvious.

All of target parts were voxelized with the aid of the python package stl-to-voxel[6]. The input to the model was a `input.npy` file holding the input voxel as a 3D binary numpy array. The output to the model was a `output.step` file. STEP (Standard for the Exchange of Product Data) files are a common universal parametric design file. STEP file creation was accomplished with the aid of the scripting-based CAD library cadquery[7].

C. Methods and Algorithms

The size of $|S|$ is alarmingly large. For example, a simple $10 \times 10 \times 10$ voxel will have 2^{1000} states. For this project, much finer voxels with larger dimensions than $10 \times 10 \times 10$ were used to preserve object resolution. Clearly, due to the sheer size of $|S|$, it is computationally impractical to find an exact reward model for all states. Additionally, due to the variability of the target state voxel, an offline approach is inappropriate because the underlying 'dynamics' of this problem are case-specific. State-action-reward histories from a previous target part are useless to learning the policy for a new target part. Thus, a tailored online method was used to learn the optimal policy.

1. Action Space Simplifications and Rollout

In order to implement a rollout strategy, I reduced the dimensionality of $|A|$. As-defined, an action can be broken down into a sequence of sub-actions, where each sub-action decides on only the next parameter. Intuitively, the first sub-action in this sequence is to decide which coordinate to make the centroid of the next primitive geometry. My lower-dimension action space contains only the freedom to decide which coordinate to grow the next centroid from. The set of possible coordinates is the set of all coordinates in s_t remaining to be occupied by s . Notice the size of $|A|$ is dynamic and depends heavily on the current state s .

Once a centroid is decided on, forward-search is used as a rollout strategy. All possible permutations of sub-actions are tried from the centroid and an optimal geometry yielding max rewards is found. Searching over every possible combination of possible geometries is a still a computationally demanding task. For example, if there are n possible values for the x, y, and z dimensions of a box, there are n^3 boxes that can be rolled out. To reduce the computational demand of rollouts, algorithm 1 was implemented [Appendix 1]. Figure 3 demonstrates a 2D application of this algorithm to finding an optimal box.

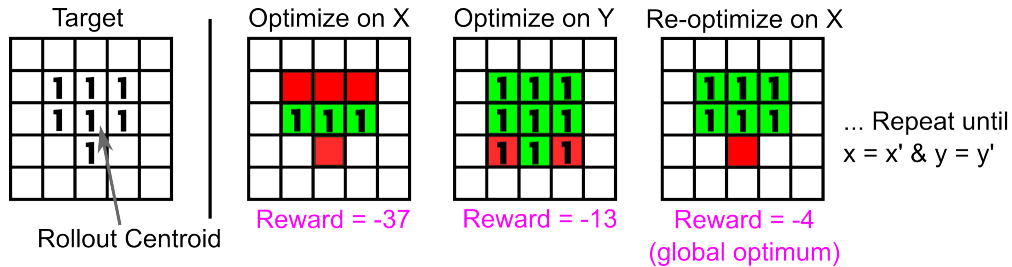


Fig. 3 2D example of rollouts of boxes from a selected centroid. $\beta = -10$

2. Exploration Strategy

The model needs a systematic way of determining which centroid to try next. A custom exploration strategy was implemented that leverages the intuition that the best locations to look for centroids are the center-of-masses of remaining mass-clusters to be occupied in the s_t . A mean-clustering algorithm was implemented [Appendix 2] that identifies the remaining unoccupied mass-clusters in s_t and finds each cluster's mean. The mean of the largest cluster is chosen as the next centroid to explore. If the mean of the largest cluster is a coordinate that itself is unoccupied in s_t , then a random occupied coordinate is explored. This case can occur for mass-clusters that resemble a donut for example. Figure 4 demonstrates a 2D example of this exploration strategy.

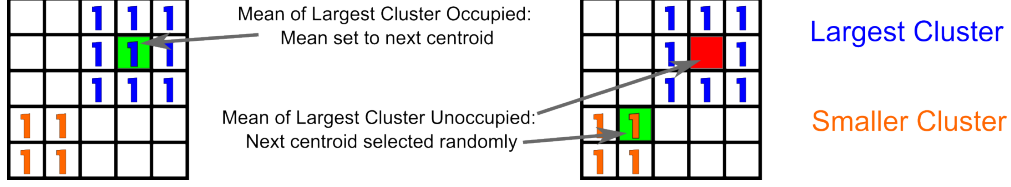


Fig. 4 2D example of joint Mean-Clustering and Random Exploration Strategy

3. Policy Valuation, Iteration, and Pruning

Recall that a policy π is a sequence of actions that yields a final state voxel s_f . I define a hyper-parameter α as a stop-criteria for growing π . α is defined as the proportion of s_t that has been occupied. For example, if $\alpha = 0.95$, then π will stop growing once 95% of s_t is also occupied by s_f .

The value of a policy π from any state s is denoted U^π and defined recursively as follows:

$$U^\pi(s) = R(s) + \gamma U^\pi(s'|a') \quad (1)$$

This definition is a flavor of the Bellman Equation. γ is the discount factor hyper-parameter that reduces the reward of actions taken later in the policy, encouraging the model to maximize rewards earned through minimal actions: a crucial feature to design efficiency.

The initial epoch of policy iteration will learn a policy based solely off of centroids chosen by the exploration strategy. Subsequent epochs greedily optimize and prune the existing policy. Policy iteration [Appendix C] takes the centroid of the first action in the previous policy and runs 1 step of 3D Hooke-Jeeves gradient ascent on the centroid. If the optimal action rolled-out from a neighboring centroid yields greater rewards than the current object rolled-out from the current centroid, the action from the neighboring centroid serves as the replacement action in the new policy. Figure Z demonstrates a single 2D iteration of a policy update with circle-creating actions.

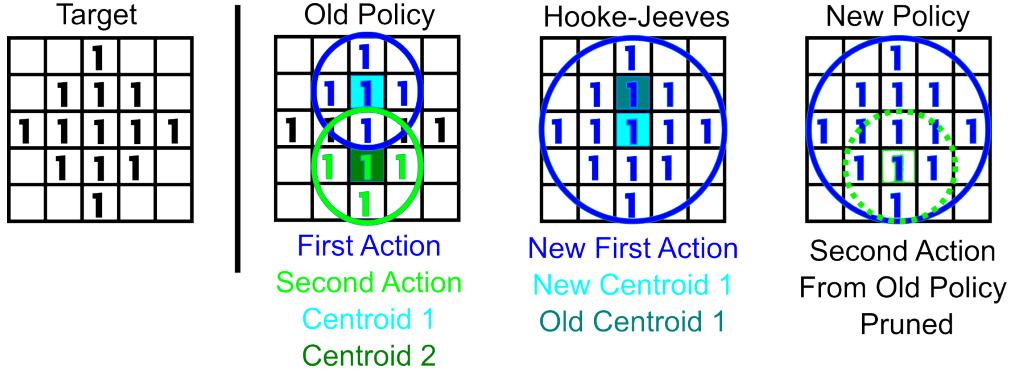


Fig. 5 Single policy update. The first action is optimized via Hooke-Jeeves gradient ascent. The new, improved first action already occupies what was the centroid of the second action, so the second action is pruned.

For subsequent actions in the old policy, the policy iteration algorithm checks whether or not the updated sequence of preceding actions in the new policy result in a state that already occupies the centroid of the action. If the centroid is already occupied, the action is deemed unnecessary and is pruned from the policy.

After updating a policy, it is possible that the new policy no longer meets the stop-criteria α due to pruned actions. The exploration algorithm is then called upon to add actions until α is met again.

Finally, equation 2 defines policy iteration convergence.

$$\frac{U^{\pi'}(s_i) - U^\pi(s_i)}{U^\pi(s_i)} \leq \epsilon \quad (2)$$

Equation 2 states that a policy is converged after the percent change in value-functions drops below a certain hyper-parameter threshold ϵ .

D. Results

1. Results on Evaluation Data

I begin by discussing the results of the RL model when learning how to make the 'easy', 'medium', and 'hard' target objects with a fixed set of hyper-parameters ($\beta = -10$, $\alpha = 0.95$, $\gamma = 0.95$, and $\epsilon = 0.01$).

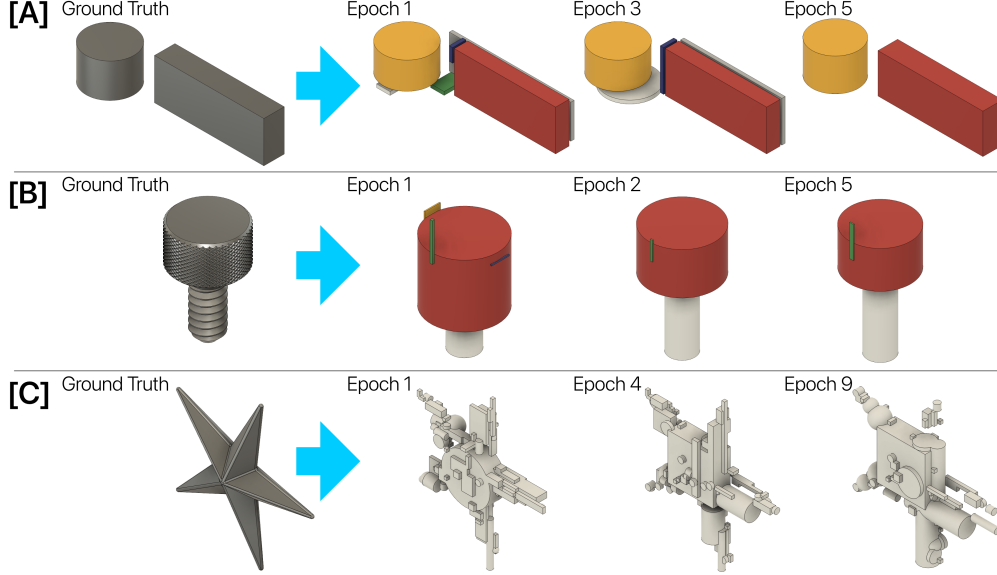


Fig. 6 Easy, Medium, and Hard target objects at different stages of training epochs.

Figure 6 illustrates the policy evolution of the model for the three target objects. Qualitatively, it is evident that the policy iteration algorithm converges to a final state s_f that closer and closer resembles s_t across multiple epochs. Less obvious is the increase in the rewards of earlier actions and the reduction in the number of actions in π across epochs. Table 1 reports these efficiency metrics.

Table 1
Evolution of key efficiency metrics

Target Object	Metric	First Epoch	Last Epoch
Easy	$R(s(a_1))$	-18129	-18129
	# Actions in π	6	2
Medium	$R(s(a_1))$	-553491	-222864
	# Actions in π	5	3
Hard	$R(s(a_1))$	-18499	-18037
	# Actions in π	91	54

Firstly, observe that $R(s(a_1))$ always increases between the first and last epoch while # Action in π always decreases. This is a positive indicator of learned design efficiency. Observe for the 'easy' object, $R(s(a_1))$ stays constant throughout all epochs due to smart-mean-clustering exploration. The centroid of the box is immediately identified as a_1 and the optimal box is immediately found. Observe for the 'medium' object, $R(s(a_1))$ between the first and last epoch increases significantly. This occurs as the centroid of a_1 eventually converges to the centroid of the head of the target screw (visually indicated by the evolution of the red cylinder). Finally, observe for the 'hard' object, while $R(s(a_1))$ does not change significantly, there is a significant decrease in the # of Actions in π .

This particular observation is also visually supported. Figure 7 shows that the model seems to learn the most efficient way to create various arms of the star using available primitive actions. For the two arms on the left [B] which are unaligned with any cardinal axes, the model learns to create efficiently using a series of shrinking spheres. On the other hand, for two arms on the right [A] which are closely aligned with a cardinal axis, the model learns to create efficiently

using primarily a single cylinder aligned with a cardinal axis. The smaller peripheral geometries seen in both Figure 6 and 7 are the result of minute actions being taken with small marginal rewards in order to satisfy the policy stop-criteria α .

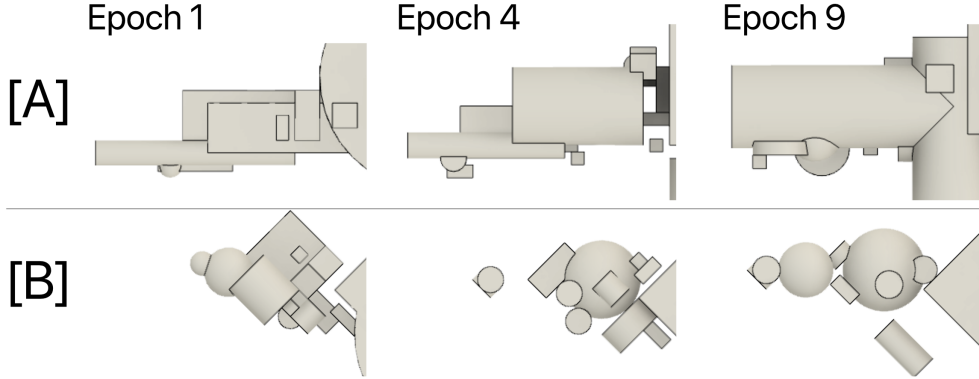


Fig. 7 Policy evolution over 9 epochs of learning on two different legs of the 'hard' star-shaped object. A clear dominant policy on efficient leg design emerges.

2. Hyper-parameter Effects Analysis

α simply measures how much of s_t needs to be filled for our final state s_f to be considered complete. Ideally, $\alpha = 1$ as that means the final object must contain at least the entirety of the target object. However, in practice, this leads to drastic extensions in convergence time and may result in extreme over-fitting with tiny primitive bodies seeking to occupy remaining unoccupied spaces.

ϵ is also a simple parameter that determines how similar two successive learned policies must be in order for policy iteration to be considered converged. Ideally, $\epsilon = 0$ as that means the successive policies were identical and policy search has converged to at least a local optima. However, in practice, this also leads to drastic extensions in convergence time.

β and γ are trickier parameters that are somewhat co-dependent and have a large impact on the final object produced.

- For $\gamma \rightarrow 0$ and $\beta \ll -1$: The model prioritizes obtaining as much reward as possible on earlier actions, yet it also wants to fit the target object as tightly as possible. This leads to conflicting interests in the model.
- For $\gamma \rightarrow 0$ and $\beta \rightarrow -1$: The model prioritizes obtaining as much reward as possible on earlier actions, and it also allows for looseness in how nicely an action needs to fit the target object. This leads to agreeing interests in the model and a low number of loose-fitting actions.
- For $\gamma \rightarrow 1$ and $\beta \rightarrow -1$: The model does not discount an extended number of actions and is only focused on fitting the model well, yet also allows for looseness in how nicely an action needs to fit the target object. This leads to conflicting interests in the model.
- For $\gamma \rightarrow 1$ and $\beta \ll -1$: The model does not discount an extended number of actions and is only focused on fitting the model well, and it also wants to fit the target object as tightly as possible. This leads to agreeing interests in the model and a very high number of tight-fitting actions.

IV. Conclusion

I present PDL: Parametric Design Learning as an application of reinforcement learning techniques to parametric 3D object creation. To do so, I synthesize 'policies' or a sequence of parametric actions necessary to modify a discretized 3D space, or voxel, in order to match a target voxel as closely as possible. The entire problem can be captured as a Markov-Decision Process (MDP) with an appropriate exploration strategy, rollout strategy, and policy search algorithm implemented with heuristics of the specific problem in mind.

Appendix

A.

Algorithm 1 Optimized Rollout Algorithm

```

 $x \leftarrow 1, y \leftarrow 1, z \leftarrow 1$  ▷ Initialize parameters
 $x' \leftarrow 0, y' \leftarrow 0, z' \leftarrow 0$ 
while  $(x, y, z) \neq (x', y', z')$  do
     $x' = \arg \max_x R(s'(a(x, y, z)))$ 
     $y' = \arg \max_y R(s'(a(x', y, z)))$ 
     $z' = \arg \max_z R(s'(a(x', y', z)))$ 
end while
Return:  $x, y, z$  ▷  $x, y, z$  used for boxes. replace with  $r, h, axis$  for cylinders

```

B.

Algorithm 2 Mean-Clustering Exploration Algorithm

```

cluster_means, cluster_sizes = mean_clustering( $s$ )
 $s_r = (s \cup s_t) - s$  ▷  $\cup$  indicates a logical or
max_mean = cluster_means[arg max(cluster_sizes)]
if  $s_r[\text{max\_mean}] == 1$  then
    Return: max_mean ▷ Return the mean of the largest cluster
else
    Return: random_coordinate ▷ Return a random occupied coordinate in target_remaining
end if

```

C.

Algorithm 3 Policy Iteration

```

Input:  $\pi_i = [a_1, a_2, \dots, a_n]$ 
 $\pi_{i+1} = []$  ▷ Initialize new policy
for  $a_i$  in  $\pi_i$  do
     $s_f = \text{execute}(\pi_{i+1})$  ▷ execute returns final state after following actions in policy
     $s_r = (s_f \cup s_t) - s_f$ 
    if  $s_r[\text{centroid}(a_i)] == 1$  then
         $a' = \text{Hooke\_Jeeves}(a_i)$ 
         $\pi_{i+1}.\text{append}(a')$ 
    end if
end for

```

References

- [1] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, & Ren Ng. (2021). NeRF: representing scenes as neural radiance fields for view synthesis. *Communic. ACM*, 65(1), 99–106. <https://doi.org/10.1145/3503250>
- [2] Poole, B., Jain, A., Barron, J. T., & Mildenhall, B. (2022). DreamFusion: Text-to-3D using 2D Diffusion. *ArXiv*. <https://arxiv.org/abs/2209.14988>
- [3] Lin, C., Gao, J., Tang, L., Takikawa, T., Zeng, X., Huang, X., Kreis, K., Fidler, S., Liu, M., & Lin, T. (2022). Magic3D: High-Resolution Text-to-3D Content Creation. *ArXiv*. <https://arxiv.org/abs/2211.10440>
- [4] Koch, S., Matveev, A., Jiang, Z., Williams, F., Artemov, A., Burnaev, E., Alexa, M., Zorin, D., & Panozzo, D. (2018). ABC: A Big CAD Model Dataset For Geometric Deep Learning. *ArXiv*. <https://arxiv.org/abs/1812.06216>
- [5] Wu, R., Xiao, C., & Zheng, C. (2021). DeepCAD: A Deep Generative Network for Computer-Aided Design Models. *ArXiv*. <https://arxiv.org/abs/2105.09492>
- [6] Pederkoff, Charles, stl-to-voxel, (2022), GitHub repository, <https://github.com/cpederkoff/stl-to-voxel>
- [7] Oberthanner, Thomas, cadquery, (2023), GitHub repository, <https://github.com/CadQuery/cadquery>